

Adaptive Routing and Patch Tournaments for Agentic Program Repair

Extending the Failure Recovery System

Meet Patel (23BCE177) Vedant Mehta (23BCE180)

School of Technology, Institute of Technology, Nirma University, Ahmedabad

Guide: Mr. AjayKumar Patel

Outline

- 1 Motivation
- 2 Related Work & Novelty
- 3 Extension 1: Adaptive Routing Controller
- 4 Extension 2: Patch Tournament
- 5 What's Actually New

Where We Left Off

- Original system: 6-agent pipeline vs. single-prompt baseline, evaluated on 3 benchmarks $\times$ 2 backends.
- **Central finding:** multi-agent's benefit over baseline is *conditional* on model capability and bug structure — not universal.

Benchmark (Gemini)	Multi-Agent	Baseline
QuixBugs	77.4%	74.2%
HumanEvalFix	68.3%	66.5%
DebugBench	78.2%	94.1%

The Problem

A system that sometimes needs 1 LLM call and sometimes needs up to 12, with no way to know which in advance, always pays worst case.

Two Extensions, Straight From Our Own Future Work

Our original report's Future Work explicitly proposed:

- ① “An adaptive controller that routes a bug to multi-agent or single-prompt repair based on a cheap upfront complexity estimate”
- ② “Debate and consensus mechanisms” among candidate fixes

This work **implements both**, as real, runnable additions to the existing codebase.

Is This Already a Solved Problem?

Routing (Extension 1)

- FrugalGPT: cascade cheap→expensive LLM by learned quality check
- RouteLLM: binary win-predictor routes weak vs. strong model
- *Neither is APR-specific; neither routes single-prompt vs. multi-agent **pipeline***

Tournaments (Extension 2)

- CodeT: sample code + tests, pick by dual execution agreement
- AlphaRepair, ChatRepair: multi-candidate, test-validated patches
- *Selection is sequential/ranked, not a parallel tournament with a reported disagreement signal*

Novelty Claim (honest version)

Both mechanisms have precedent individually. We found no prior system combining a bug-feature-trained router *and* a test-validated patch tournament inside one sequential multi-agent APR pipeline, evaluated across dual cloud/local backends and 3 benchmarks.

Design: Route Before You Spend a Single LLM Call

Constraint

The predictor must use features computable with **zero LLM calls** — otherwise the "savings" are fake.

Features (`features.py`): `loc`, `chars`, cyclomatic complexity, `#tests`, `#functions`, `dataset/language/test-style`

Key feature: `buggy_pass_rate` — % of tests the *unmodified* buggy code already passes.
Free: the coordinator already runs this test pass once per bug.

Labels, Model, and Leak-Free Evaluation

- Label: `needs_multiagent` = 1 iff multi-agent succeeded **and** baseline failed (12.5% positive rate, 303 problems)
- Model: Random Forest (300 trees, depth 5, balanced weights) — chosen over logistic regression, which barely beat the base rate
- **Evaluation:** stratified 5-fold CV — router trained on 4 folds, routes held-out fold's bugs, scored against each bug's *already-measured* historical outcome

Why this is honest

Zero new LLM calls to "evaluate," zero train/test leakage — standard offline policy evaluation.

Results: Router vs. Fixed Strategies

Strategy	Success Rate	Avg. LLM Calls/Bug
Always Baseline	56.4%	1.00
Always Multi-Agent	56.8%	7.91
Adaptive Router	57.8%	1.39
Oracle (best of both)	69.0%	—

- Router matches/exceeds **both** fixed strategies' success rate
- At **5.7**× fewer LLM calls than always-multi-agent
- Oracle gap (69.0%) shows real headroom still unrouted

Results: It's Not a Uniform Win

Dataset	Baseline	Multi-Agent	Routed	Oracle
QuixBugs	58.1%	51.6%	54.8%	77.4%
HumanEvalFix	53.0%	54.3%	53.0%	62.2%
DebugBench	61.1%	62.0%	65.7%	76.9%

Honest finding

Router clearly wins on DebugBench, but loses to plain baseline on QuixBugs (small sample, more algorithmic bugs). This *echoes* the original paper's conditional-benefit thesis rather than resolving it.

Design: Stop Betting on One Stochastic Draw

- TournamentCoordinator subclasses the original coordinator — Agents 1–3 (Detect/Localize/Debug) run **once** per attempt, unchanged
- Agent 4 (Patch Generation) resampled $K=3\times$ at temperatures $[0.2, 0.5, 0.9]$ from the *same* diagnosis
- Every candidate validated (Agents 5–6); keep first full-pass candidate, else best partial
- Free byproduct: **agreement score** — % of candidate pairs that produced identical code (1.0 = full consensus, 0.0 = all differ)

Cost

$3 + K$ LLM calls/attempt vs. original 4 — $1.5\times$ overhead at $K=3$, paid only where stochastic variance is highest.

Results: All Three Benchmarks (131 bugs total)

	Baseline	Multi-Agent	Tournament
QuixBugs ($n=31$)	58.1%	51.6%	64.5%
HumanEvalFix ($n=50$)	76.0%	68.0%	72.0%
DebugBench ($n=50$)	58.0%	62.0%	70.0%
Aggregate ($n=131$)	64.9%	61.8%	69.5%

- Tournament beats baseline on 2/3 benchmarks **and** in aggregate — notably including DebugBench, where the original paper's Gemini results had baseline decisively winning
- Cost: $1.55\times$ LLM calls/bug (matches $1.5\times$ nominal prediction) — but wall-clock **mean** overhead is $6.3\times$, **median** only $1.8\times$ (a few timeout-heavy infinite-loop bugs dominate the mean)

Agreement vs. Outcome: Not a Universal Law

	Overall	On Success	On Failure
QuixBugs	0.594	0.562	0.651
HumanEvalFix	0.700	0.722	0.643
DebugBench	0.482	0.536	0.356

- HumanEvalFix & DebugBench: intuitive direction — high agreement correlates with success
- **QuixBugs inverts this** — high agreement correlates with *failure* (model confidently wrong on all 3 draws; likely linked to QuixBugs' higher InfiniteLoop/RuntimeError rate)

Honest takeaway

We drafted the "surprising universal pattern" from QuixBugs alone, then HumanEvalFix/DebugBench contradicted it. Agreement is informative but benchmark-conditional, not a fixed law.

What Changed vs. the Original System

- ① **New decision layer** above the existing architecture — a trained classifier now picks baseline vs. multi-agent per bug, where previously the experimenter picked one for an entire run
- ② **New inside-step mechanism** — Agent 4 goes from one stochastic draw to a validated tournament, targeting failures caused by sampling variance rather than bad diagnosis
- ③ **New signal that didn't exist before** — tournament agreement as a per-bug confidence/uncertainty estimate

All three are *additive*: zero changes to the original 6 agents, the retry loop, or the dataset-agnostic execution layer.

Limitations

- Router recall on the rare "needs multi-agent" class is low (0.132) — tuned for net cost/success, not for catching every such bug
- Training data is the original paper's own 303 problems, not an independent population — router has only ever seen 3 benchmarks
- Tournament's $1.5\times$ per-attempt overhead only pays off where extra samples find a passing patch single-draw would miss
- Both extensions evaluated on the local backend only — not re-validated on Gemini

Conclusion & Future Work

- Implemented both extensions proposed in our own prior future work, as real runnable system components
- Adaptive router: matches/exceeds fixed strategies at $5.7\times$ lower cost, honestly reported as benchmark-dependent, not a uniform win
- **Patch tournament: the stronger result** — 69.5% aggregate vs. 61.8% (multi-agent) and 64.9% (baseline) across 131 bugs, beating baseline outright on 2/3 benchmarks including DebugBench, at near-nominal LLM-call cost

Next: three-tier routing (baseline / multi-agent / multi-agent+tournament), per-candidate timeout budgets, non-Python router features, fix the DebugBench naming collision upstream, Gemini re-validation.

Thank You

Questions?